Great question. The documentation is a little confusing at first.

The key part is the function signature:

func fmt.Errorf(format string, a ...any) error

Notice that it **returns an `error`, not a `string`**.

So why does the documentation say:

> "returns the string as a value that satisfies error"

What it means is:

1. `fmt.Errorf` first formats a string (just like `fmt.Sprintf`).
2. Then it wraps that string inside an object that implements the `error` interface.
3. Finally, it returns that `error`.

For example:

err := fmt.Errorf("user %d not found", 42)

fmt.Println(err)

Output:

user 42 not found

Here,

err

is **not a string**.

Its type is:

fmt.Printf("%T\n", err)

Output:

*errors.errorString

Compare with `fmt.Sprintf`

`fmt.Sprintf` returns a **string**.

msg := fmt.Sprintf("Hello %s", "Suyash")

fmt.Printf("%T\n", msg)

Output:

string

Compare with `fmt.Errorf`

err := fmt.Errorf("Hello %s", "Suyash")

fmt.Printf("%T\n", err)

Output:

*errors.errorString

Even though printing it displays:

Hello Suyash

the value is an **error**, not a string.

What about `%w`?

This is one of the most useful features.

Suppose you receive an error:

err := errors.New("database connection failed")

Instead of doing:

return fmt.Errorf("creating user failed: %v", err)

you should do:

return fmt.Errorf("creating user failed: %w", err)

Now Go remembers the **original error**.

Example:

original := errors.New("database error")

wrapped := fmt.Errorf("failed to create user: %w", original)

fmt.Println(wrapped)

Output:

failed to create user: database error

Later you can recover the original error:

fmt.Println(errors.Unwrap(wrapped))

Output:

database error

or check it:

```
if errors.Is(wrapped, original) {
   fmt.Println("It's the same underlying error!")
}
```

Summary

- `fmt.Sprintf(...)` → returns a **string**.
- `fmt.Errorf(...)` → formats a string, **creates an error containing that string**, and returns an **error**.
- `%v` includes an error's text.
- `%w` includes the error **and preserves it for wrapping/unwrapping** with `errors.Is`, `errors.As`, and `errors.Unwrap`.

Once you start building Go backends, you'll use `fmt.Errorf("...: %w", err)` constantly because it's the idiomatic way to add context while preserving the original error.

That's a great question. This is one of the core ideas in Go, and understanding it will make your backend code much cleaner.

Suppose you have:

storage, err := sqlite.New(cfg)

where `sqlite.New(cfg)` returns something like:

func New(cfg config.Config) (*Storage, error)

So `storage` is of type:

*sqlite.Storage

Now imagine your HTTP server constructor looks like:

```
func New(storage storage.Storage) *Server {
   return &Server{
      storage: storage,
   }
}
```

Notice it accepts:

storage.Storage

which is an **interface**, not `*sqlite.Storage`.

Why not just accept `*sqlite.Storage`?

If you wrote:

func New(storage *sqlite.Storage)

then your server is tightly coupled to SQLite.

Tomorrow, suppose you decide to use PostgreSQL:

postgres.New(cfg)

or even an in-memory database for testing:

memory.New()

You would have to change the server code everywhere.

Instead, define an interface:

```
type Storage interface {
    SaveStudent(...)
    GetStudent(...)
    DeleteStudent(...)
}
```

Then make SQLite implement it:

```
type Storage struct {
    db *sql.DB
}

func (s *Storage) SaveStudent(...) { ... }
func (s *Storage) GetStudent(...) { ... }
func (s *Storage) DeleteStudent(...) { ... }
```

Now this works automatically:

```
storage, _ := sqlite.New(cfg)

server := New(storage)
```

because `*sqlite.Storage` implements the `Storage` interface.

Go does this **implicitly**. You don't write:

```
type Storage struct implements Storage
```

like you would in Java. If the methods match, Go treats it as implementing the interface.

Think of it like this

The server doesn't care **how** data is stored.

It only says:

> "Give me something that can save, fetch, and delete students."

SQLite says:

> "I know how."

PostgreSQL says:

> "I know how too."

An in-memory implementation says:

> "I can do that as well."

The server never changes.

A real-world analogy

Imagine a phone charger.

The phone expects:

> "A USB-C charger."

It doesn't care whether it's from Apple, Samsung, or Anker.

The **USB-C connector** is the interface.

The charger brand is the concrete implementation.

Similarly:

- `storage.Storage` → the contract (interface)
- `*sqlite.Storage` → one implementation
- `*postgres.Storage` → another implementation

The server depends on the **contract**, not the implementation.

One question for you to check your understanding:

If tomorrow you create:

```
type MemoryStorage struct {
    students map[int]Student
}
```

and implement all the methods required by the `Storage` interface, would you need to change the `New(storage storage.Storage)` function signature?

Or could you simply do:

```
mem := memory.New()
server := New(mem)
```

What do you think the answer is?

😂 This is a very common point of confusion when you start working with databases in Go. Let's break it down.

You have:

CreateStudent(name string, email string, age int) (int64, error)

Your question is:

> **Why is the return type `int64` instead of `int`?**

Short answer

Because **database IDs are usually represented as 64-bit integers** by Go's `database/sql` package.

Imagine this table

```
CREATE TABLE students (
   id INTEGER PRIMARY KEY AUTOINCREMENT,
   name TEXT,
   email TEXT,
   age INTEGER
);
```

When you insert a student:

```
INSERT INTO students(name, email, age)
VALUES ('Suyash', 'abc@gmail.com', 22);
```

SQLite automatically generates:

id	name
1	Suyash

Now in Go, after inserting, you usually write:

```
result, err := db.Exec(query, name, email, age)
```

Then:

```
id, err := result.LastInsertId()
```

Here's the important part:

```
func (r Result) LastInsertId() (int64, error)
```

Notice **Go itself returns an `int64`**.

So naturally your function also returns:

func CreateStudent(...) (int64, error)

because you're just passing that ID back.

Why didn't Go use `int`?

Because `int` depends on the machine.

On a 32-bit machine:

int = 32 bits

Maximum value:

2,147,483,647

On a 64-bit machine:

int = 64 bits

Maximum value:

9,223,372,036,854,775,807

Databases don't want IDs that depend on the CPU architecture. They use a fixed width.

That's why Go's SQL package standardizes on `int64`.

Why not `int32`?

Imagine your website becomes huge.

Student ID

1
2
3
...
2,000,000
...
500,000,000
...
5,000,000,000

`int32` would overflow.

`int64` gives an enormous range, so practically you won't run out of IDs.

In your code

Suppose:

```
func CreateStudent(...) (int64, error) {
    result, err := db.Exec(...)
    if err != nil {
        return 0, err
    }

    id, err := result.LastInsertId()

    return id, err
}
```

If you insert:

Suyash

then

id = 1

Second insert:

Rahul

id = 2

The values look small, but their **type** is still `int64`.

Think of it like this

The number:

5

can be stored as:

int8
int16
int32
int64
int

The **value** is the same (`5`).

The **type** tells the computer how much space to reserve.

So:

var a int = 5
var b int64 = 5

Both represent the value 5, but `b` uses a 64-bit integer type.

The key takeaway

Your function returns:

(int64, error)

because it is returning the ID produced by the database, and the `database/sql` package's `LastInsertId()` method already returns an `int64`. Using `int64` keeps the type consistent and avoids architecture-dependent behavior.

This is a good question to ask—understanding **why `int`, `int64`, and `uint` exist and when to use each** is fundamental in Go. Once that clicks, many database APIs will make much more sense.

This is one of the most important pieces of Go database programming. Let's go through it **line by line** as if we're the computer.

Step 1: You call

student, err := storage.GetStudentById(5)

So you want the student whose **id = 5**.

Step 2: Prepare()

```
stmt, err := s.Db.Prepare(`
    SELECT id, name, email, age
    FROM students
    WHERE id = ?
    LIMIT 1
`)
```

What is `Prepare()`?

Think of it as **creating a SQL template**.

You're **not executing** the SQL yet.

You're telling SQLite:

> "Hey SQLite, keep this query ready. Later I'll give you the value for ?."

So internally SQLite remembers something like:

```
SELECT id,name,email,age
FROM students
WHERE id = ?
LIMIT 1
```

Notice the ?.

It is a placeholder.

Nothing has happened yet.

Imagine ordering pizza.

Instead of saying

One Veg Pizza

you say

One ____ Pizza

Later you fill in

One Paneer Pizza

The blank is exactly like ?.

Why Prepare?

It has two advantages.

1. Security

Suppose a user enters

5 OR 1=1

If you build SQL using string concatenation,

query := "SELECT * FROM students WHERE id=" + input

the SQL becomes

SELECT * FROM students WHERE id=5 OR 1=1

which is dangerous.

Prepared statements separate SQL from data, preventing this kind of injection.

---

2. Performance

If the same query runs many times,

SQLite parses it once and can reuse it.

Step 3

defer stmt.Close()

Once this function finishes,

Prepared Statement

gets destroyed.

Otherwise resources remain allocated.

Step 4

var student types.Student

Currently

student

Id   = 0
Name = ""
Email = ""
Age   = 0

An empty struct.

Step 5

This is the most important line.

```
err = stmt.QueryRow(id).Scan(
    &student.Id,
    &student.Name,
    &student.Email,
    &student.Age,
)
```

Let's split it.

First

stmt.QueryRow(id)

Remember the SQL?

```
SELECT id,name,email,age
FROM students
WHERE id = ?
LIMIT 1
```

You passed

id

Suppose

id = 5

SQLite replaces

?

with

5

So the executed query becomes

```
SELECT id,name,email,age
FROM students
WHERE id = 5
LIMIT 1
```

SQLite searches the table.

Suppose the table contains

id	name	email	age
5	Suyash	abc@gmail.com	22

It finds exactly one row.

That row is temporarily held inside the returned `Row` object.

Nothing has been copied into your struct yet.

Now comes Scan()

Scan(...)

Think of `Scan()` as saying:

> "Take each column from the database row and copy it into these Go variables."

Database row:

5
Suyash
abc@gmail.com
22

Your code says

```
Scan(
   &student.Id,
   &student.Name,
   &student.Email,
   &student.Age,
)
```

So Go performs:

student.Id = 5

student.Name = "Suyash"

student.Email = "abc@gmail.com"

student.Age = 22

Now your struct becomes

```
student := Student{
    Id: 5,
    Name: "Suyash",
    Email: "abc@gmail.com",
    Age: 22,
}
```

Why use &?

Because `Scan()` needs to **modify your variables**.

If you wrote

Scan(student.Id)

you're giving it a copy of the value.

It cannot change your struct.

By writing

&student.Id

you're giving it the **memory address** where it should store the value.

You can think of it as:

Database

5 ----------> memory of student.Id

"Suyash" -----> memory of student.Name

"abc@gmail.com" -> memory of student.Email

22 ----------> memory of student.Age

Why must the order match?

Your SQL returns:

SELECT id, name, email, age

So SQLite returns

Column 1 → id
Column 2 → name
Column 3 → email
Column 4 → age

Your `Scan()` must follow the same order:

```
Scan(
    &student.Id,
    &student.Name,
    &student.Email,
    &student.Age,
)
```

If you accidentally wrote:

```
Scan(
    &student.Name,
    &student.Id,
    &student.Email,
    &student.Age,
)
```

Go would try to put

id

into a string,

and

name

into an integer,

leading to a scan/type conversion error.

What if no row exists?

Suppose you search

GetStudentById(100)

but the table has only

1
2
3
4
5

SQLite finds nothing.

Then

Scan(...)

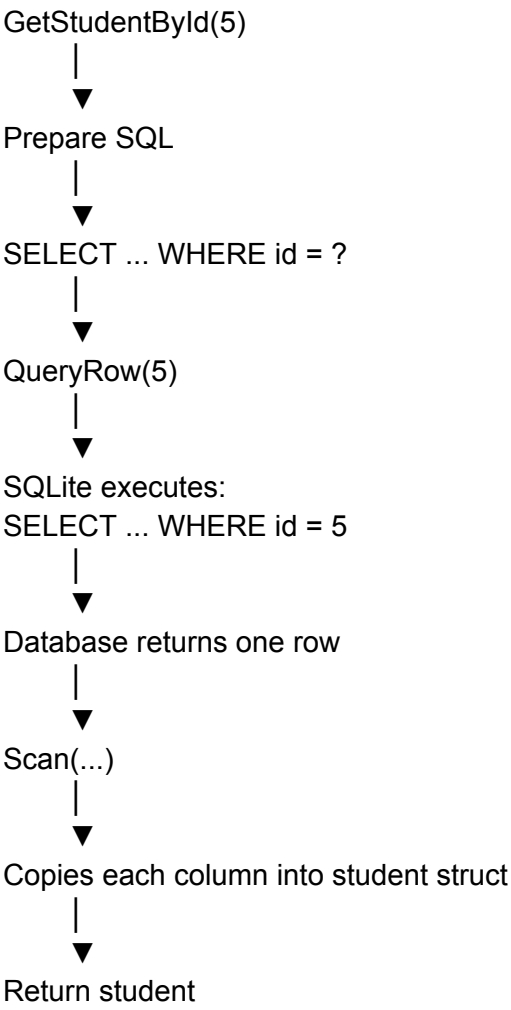
returns

sql.ErrNoRows

That's why you check:

```
if err == sql.ErrNoRows {
    ...
}
```

The complete flow



One improvement

In this function, you're calling `Prepare()` only to execute the statement **once**. That's perfectly valid, but it's not always necessary. The `database/sql` package lets you write the same logic more simply:

```
err := s.Db.QueryRow(
    `SELECT id, name, email, age FROM students WHERE id = ? LIMIT 1`,
    id,
).Scan(&student.Id, &student.Name, &student.Email, &student.Age)
```

`QueryRow` internally prepares and executes the statement for this one call. Explicit `Prepare()` is most useful when you plan to execute **the same SQL many times** with different values. That's why you'll see both styles in real Go code.

Brother, this is **the** confusing part of `database/sql` for almost everyone. Once you understand the flow, it becomes very logical.

Let's forget Go for a minute.

Imagine you go to a library.

- You ask the librarian: "Give me the book with ID 5." → One book comes back.
- You ask: "Give me all Go books." → A whole shelf of books comes back.

The database works exactly the same way.

There are three important things

1. Prepare()

Think of it as **writing the question on paper**.

```
stmt, err := db.Prepare(
   "SELECT id,name,email,age FROM students"
)
```

Nothing has happened yet.

You're just creating a reusable SQL statement.

Imagine this:

SQLite

Question:

SELECT ...
FROM students

That's all.

No data has been fetched.

---

2. QueryRow()

Used when **you expect exactly ONE row**.

Example:

SELECT * FROM students WHERE id = 5;

Only one student can have ID = 5.

So Go gives you

row := stmt.QueryRow(5)

Notice

QueryRow

↓

One row

Then

row.Scan(...)

copies that one row into your struct.

---

3. Query()

Used when **you expect MANY rows**.

Example

SELECT * FROM students;

Suppose your table is

id	name
1	Ram
2	Shyam
3	Suyash

Now the database returns

Row1
Row2
Row3

Go cannot put all of them into one struct.

Instead it returns a `Rows` object.

rows, err := stmt.Query()

Think of `rows` like a **cursor**.

What is a Cursor?

Imagine a finger pointing at the result.

Initially

Before first row

↓

---
1 Ram

2 Shyam
3 Suyash

The cursor is **before** the first row.

It hasn't read anything.

Now you call

rows.Next()

The cursor moves.

↓

1 Ram
2 Shyam
3 Suyash

Now row 1 is available.

Then

rows.Scan(...)

copies

1
Ram

into

student

Then loop again.

rows.Next()

Cursor moves.

1 Ram

↓

2 Shyam

3 Suyash

Again

rows.Scan(...)

copies

2
Shyam

into your struct.

Again

rows.Next()

1 Ram
2 Shyam

↓

3 Suyash

Scan copies row 3.

Again

rows.Next()

Now there are no more rows.

End

`Next()` returns

false

The loop stops.

So why not Scan directly?

Because the database doesn't know how many rows exist.

Maybe

0 rows

Maybe

5 rows

Maybe

10 million rows

It cannot return a slice immediately.

Instead it says

> "Here's a cursor. Keep asking me for the next row."

This is memory efficient.

Why rows.Close()?

Suppose the database opened a connection.

Your App

↓

Database Connection

↓

Rows

If you never close it,

the connection remains occupied.

Eventually

Connection 1 busy

Connection 2 busy

Connection 3 busy

No more connections remain.

That's why

defer rows.Close()

is important.

It tells Go

> "After I'm done reading, release the database resources."

Why stmt.Close()?

A prepared statement also uses resources.

SQLite

Prepared Statement

Once you're done,

stmt.Close()

removes it.

Again,

defer stmt.Close()

ensures cleanup.

Was Prepare needed here?

Actually... not really.

Your code

```
stmt, _ := db.Prepare(...)
rows, _ := stmt.Query()
```

can be written simply as

```
rows, err := db.Query(
    "SELECT id,name,email,age FROM students",
)
```

Much shorter.

Similarly,

Instead of

```
stmt, _ := db.Prepare(...)
stmt.QueryRow(id)
```

you can simply write

```
db.QueryRow(
   "SELECT id,name,email,age FROM students WHERE id=?",
   id,
)
```

Most Go developers write it this way unless they need to reuse the same prepared statement many times.

When should you use Prepare()?

Suppose you're importing **10,000 students**.

Without `Prepare()`:

Loop

↓

Parse SQL

Execute

Parse SQL

Execute

Parse SQL

Execute

SQLite parses the SQL 10,000 times.

With `Prepare()`:

Prepare once

↓

Execute

Execute

Execute

Execute

Execute

The SQL is parsed once and reused, which is more efficient.

The mental model to remember

Prepare()

Create a reusable SQL statement.

QueryRow()

Execute and expect ONE row.

Query()

Execute and expect MANY rows.

Next()

Move the cursor to the next row.

Scan()

Copy the current row's columns into Go variables.

rows.Close()

Release the result set and database resources.

stmt.Close()

Release the prepared statement.

If you understand this flow, you've understood about **80% of how Go's `database/sql` package works**. The rest is mostly variations on these same concepts.

Brother, don't worry. This is one of those concepts that feels impossible at first, but once it clicks, you'll wonder why it ever seemed confusing.

Let's ignore your project for a minute.

Step 1: Suppose I have a Dog

type Dog struct{}

Dog has one method:

```
func (d Dog) Speak() {
    fmt.Println("Bark")
}
```

Now suppose I define an interface.

```
type Animal interface {
    Speak()
}
```

This interface says:

> "I don't care what animal you are. If you can `Speak()`, you're an Animal."

Notice there is **no implementation**.

Just a rule.

Step 2: Can Dog become an Animal?

Yes.

Because Dog has

Speak()

Go automatically says

Dog satisfies Animal

You NEVER write

implements Animal

like Java.

Go checks the methods.

Step 3

Now look at this function.

```
func MakeSound(a Animal) {
   a.Speak()
}
```

Notice

Animal

NOT

Dog

This function says

> "Give me **anything** that can Speak()."

Now call it.

dog := Dog{}

MakeSound(dog)

Go thinks

Dog has Speak()

↓

Dog satisfies Animal

↓

Allowed

Now your project

You have

```
type Storage interface {
    GetStudentsById(...)
```

```
    GetStudents(...)
    CreateStudents(...)
}
```

This is exactly like

```
type Animal interface {
    Speak()
}
```

It is just saying

> "If you have these three methods, I'll call you a Storage."

Now your SQLite struct

```
type Sqlite struct {
    Db *sql.DB
}
```

has

```
func (s *Sqlite) GetStudentsById(...)
```

and

```
func (s *Sqlite) GetStudents(...)
```

and

```
func (s *Sqlite) CreateStudents(...)
```

Go checks:

Does *Sqlite have GetStudentsById?

YES

Does it have GetStudents?

YES

Does it have CreateStudents?

YES

↓

Then *Sqlite satisfies Storage

That's all.

This line

storage, err := sqlite.New(cfg)

Suppose `sqlite.New()` returns

*Sqlite

So now

storage

contains

*Sqlite

Think of it like

storage

↓

*Sqlite

Now comes the confusing part

students.New(storage)

Look at its definition.

func New(storage storage.Storage)

Read it in English.

> New wants **anything that satisfies the Storage interface.**

Now ask yourself

What am I passing?

↓

*Sqlite

Does

*Sqlite

satisfy

Storage

Yes.

Therefore Go allows it.

Imagine this.

Interface:

Storage

Must know

✔ CreateStudent()

✔ GetStudent()

✔ GetStudents()

SQLite:

Sqlite

✔ CreateStudent()

✔ GetStudent()

✔ GetStudents()

Everything matches.

So Go says

Good.

You are a Storage.

This line

func New(storage storage.Storage)

is actually two different things.

The first `storage`

storage.Storage

is the **package name**.

The second

storage

is just the variable name.

Exactly like this:

package math

func Add(math int) {
}

Here

math.Add()

↑ package

Inside the function

math

↑ variable

The variable hides the package.

It's perfectly legal, although many Go developers choose a different parameter name to avoid confusion.

Then inside New()

Suppose

func New(storage storage.Storage)

gets called.

At runtime

storage

↓

*Sqlite

Even though its type is the interface, inside that interface value is the concrete `*Sqlite`.

So later

storage.CreateStudents(...)

actually becomes

(*Sqlite).CreateStudents(...)

automatically.

You don't have to tell Go.

Think of the interface as a remote control

Imagine this remote.

Buttons

Play

Pause

Stop

The TV only knows

I need something with

Play

Pause

Stop

It doesn't care if it's

- Sony TV
- Samsung TV
- LG TV

As long as the remote has those buttons.

Your handler is like the TV.

It doesn't care whether the storage is

- SQLite
- PostgreSQL
- MongoDB
- In-memory

It just knows

CreateStudents()

GetStudents()

GetStudentById()

One thing I want to correct from the explanation you pasted

It says:

> "`storage` (the parameter name) shadows the package name."

That's true, but **that's not the important part**. Beginners often get stuck there.

The important idea is:

- `storage.Storage` is the **interface type**.
- `storage` (the parameter) is a **value** whose concrete type at runtime is `*Sqlite`.

So when you call:

students.New(storage)

you're really doing:

Passing a *Sqlite value into a parameter that accepts any type implementing the Storage interface.

That is the key concept.

I have one question for you.

If tomorrow you create:

type Postgres struct{}

and you implement:

- `CreateStudents()`
- `GetStudents()`
- `GetStudentsById()`

without changing **anything** in `students.New()`, do you think this would compile?

db, _ := postgres.New(cfg)

router.HandleFunc("/students", students.New(db))

Think about it before looking it up. If you can answer **why** it works (or doesn't), you've understood interfaces.